

Sortieralgorithmen

Maya Neudorf
maya.neudorf@uni-bielefeld.de

18. Mai 2026

Inhaltsverzeichnis

1	Was ist eine Ordnung?	2
2	das allgemeine Sortierproblem	2
3	Sortieren durch sukzessive Auswahl	2
3.1	Beweis für Korrektheit des Algorithmus	3
4	Sortieren nach Schlüsseln	3
4.1	Bubblesort	3
5	Mergesort	4
5.1	Laufzeit	4
5.2	Satz zur Anzahl von Vergleichen	4
6	Quicksort	5
6.1	Laufzeit	6
6.2	die Wahl des x	6
6.3	Erwartungswert	6
7	Heapsort	7

1 Was ist eine Ordnung?

Diese Ausarbeitung befasst sich mit Sortieralgorithmen. Um zu sortieren führen wir erst einmal den Begriff einer Ordnung ein.

Definiton 1. Sei S eine Menge. Eine Relation $R \subseteq S \times S$ heißt partielle Ordnung $\preceq$, wenn folgende Eigenschaften gelten:

- (a) $(a, a) \in R$ (Reflexivität);
- (b) $((a, b) \in R \wedge (b, a) \in R) \Rightarrow a = b$ (Antisymmetrie)
- (c) $((a, b) \in R \wedge (b, c) \in R \Rightarrow (a, c) \in R$ (Transitivität)

Die Teilmengen Relation $\subseteq$ ist ein Beispiel für so eine partielle Ordnung.

Definiton 2. Eine partielle Ordnung R von S heißt totale Ordnung, wenn $(a, b) \in R$ oder $(b, a) \in R$ für alle $a, b \in S$ gilt

Beispiel für eine totale Ordnung ist die gewohnte Ordnung $\leq$ auf $\mathbb{R}$.

2 das allgemeine Sortierproblem

Um eine Liste zu sortieren suchen wir einen Algorithmus der mit folgender Eingabe und Ausgabe arbeitet:

- **Input:** eine endliche Menge S mit einer partiellen Ordnung $\preceq$
- **Output:** Berechne eine Bijektion $f: \{1, \dots, n\} \rightarrow S$ mit $f(j) \not\preceq f(i)$ für alle $1 \leq i < j \leq n$ (eine sortierte Liste)

Die naive Herangehensweise wäre einfach die $n!$ Permutationen auszuprobieren und jeweils zu testen ob die Liste geordnet ist. Dies ist natürlich sehr ineffizient.

3 Sortieren durch sukzessive Auswahl

- **Input:** eine Menge $S = \{s_1, \dots, s_n\}$; eine partielle Ordnung $\preceq$ auf S
- **Output:** eine Bijektion $f: \{1, \dots, n\} \rightarrow S$ mit $f(j) \not\preceq f(i)$ für alle $1 \leq i < j \leq n$.

```
for  $i \leftarrow 1$  to  $n$  do  $f(i) \leftarrow s_i$ 
for  $i \leftarrow 1$  to  $n$ 
  for  $j \leftarrow i$  to  $n$ 
    if  $f(j) \preceq f(i)$  then swap( $f(i), f(j)$ )
output  $f$ 
```

Praktisch würde der Algorithmus bei der Liste 3,4,2,1 also wie folgt vorgehen. Beim ersten Durchlauf, also $i = 1$, vergleicht der Algorithmus alle Elemente der

Liste und findet so das kleinste. Dieses kleinste Element in der Liste, also die 1, wird nach ganz vorne geschrieben. Nach einer Iteration sieht die Liste also so aus: 1,3,4,2. Nun bei $i = 2$ wird die 2 herausgesucht und hinter die eins geschrieben (1,2,3,4). Obwohl wir als Menschen genau sehen, dass die Liste nun geordnet ist, muss der Algorithmus noch die zwei Iterationen durchführen. Der Algorithmus vergleicht also viermal alle vier Zahlen miteinander.

Die Laufzeit beträgt also $O(n^2)$.

3.1 Beweis für Korrektheit des Algorithmus

Intuitiv ist sehr schnell klar, dass dieser Algorithmus funktioniert. Trotzdem wird dies im folgenden formell bewiesen. Wir zeigen, dass für alle $1 \leq i \leq j \leq n$ nach Iteration (i,j) folgendes gilt:

- (a) $f(k) \not\leq f(i)$ für alle $1 \leq h < i$ und $h < k \leq n$ (vor i ist die Liste sortiert)
- (b) $f(k) \not\leq f(i)$ für alle $i < k \leq j$ (i ist kleiner als alles rechts davon)

Für b) 1. Fall $f(j) \not\leq f(i)$: es wird nichts getauscht. b) gilt also 2. Fall: $f(j) \leq f(i)$, betrachte Index k mit $i < k \leq j$: für $j = k$ gilt wegen der Antisymmetrie b) nach dem swap-Befehl. für $k < j$ galt zuvor $f(k) \not\leq f(i)$ und somit $f(k) \not\leq f(j)$. Es gilt also b).

4 Sortieren nach Schlüsseln

Für schnellere Sortieralgorithmen benötigen wir eine totale Ordnung. Um diese zu erzwingen definieren wir eine Abbildung $k : S \rightarrow K$ von der Menge der zu sortierenden Elemente S zu einer Menge von Schlüsseln K mit einer totalen Ordnung.

- **Input:** eine endliche Menge S , sowie eine Funktion $k : S \rightarrow K$ und eine totale Ordnung $\leq$ auf K .
- **Output:** Berechne eine Bijektion $f : \{1, \dots, n\} \rightarrow S$ mit $k(f(j)) \leq k(f(i))$ für alle $1 \leq i < j \leq n$ (eine sortierte Liste von Schlüsseln)

Beispiele: Bucketsort, Insertion sort, Bubblesort

4.1 Bubblesort

Bei Bubblesort wird die Liste (n-1)-mal durchgegangen und die Elemente werden jeweils verglichen. Wenn ein Element gefunden wird, dass kleiner als sein Vorgänger ist, werden Vorgänger und das Element vertauscht. Bubblesort würde die Liste (3,2,4,1) also wie folgt sortieren:

1. Iteration: (2,3,1,4)

2. Iteration: (2,1,3,4)

3. Iteration: (1,2,3,4)

Dieses Verfahren ist also manchmal schneller, braucht aber für manche Instanzen trotzdem viele Vergleiche.

5 Mergesort

- **Input:** eine Menge $S = \{s_1; \dots; s_n\}$; eine durch Schlüssel induzierte partielle Ordnung $\preceq$ auf S .
- **Output:** Berechne eine Bijektion $f: \{1, \dots, n\} \rightarrow S$ mit $f(j) \not\preceq f(i)$ für alle $1 \leq i < j \leq n$.

if $|S| > 1$
then

- sei $S = S_1 \dot{\cup} S_2$ mit $|S_1| = \lfloor \frac{n}{2} \rfloor$ und $|S_2| = \lceil \frac{n}{2} \rceil$
- sortiere S_1 und S_2 (rekursiv mit Mergesort)
- Verschmelze die Sortierungen von S_1 und S_2 zu Sortierung S .

Mergesort beruht also auf dem „Divide-and-Conquer“-Prinzip. Man teilt das Ausgangsproblem in kleinere Teilprobleme, löst diese separat, und füge die Listen geordnet wieder zusammen.

5.1 Laufzeit

Mergesort sortiert n Elemente in der Laufzeit $O(n \log n)$

Beweis: Sei $c \in \mathbb{N}$ mit $T(1) \leq c$ und

$$T(n) \leq T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + cn \quad \text{für alle } n \geq 2.$$

Behauptung: $T(2^k) \leq c(k+1)2^k$ für alle $k \in \mathbb{N} \cup \{0\}$.

Induktion über k : Für $k = 0$ trivial.

Für $k \in \mathbb{N}$ gilt: $T(2^k) \leq 2 \cdot T(2^{k-1}) + c2^k \leq 2 \cdot ck2^{k-1} + c2^k = c(k+1)2^k$.

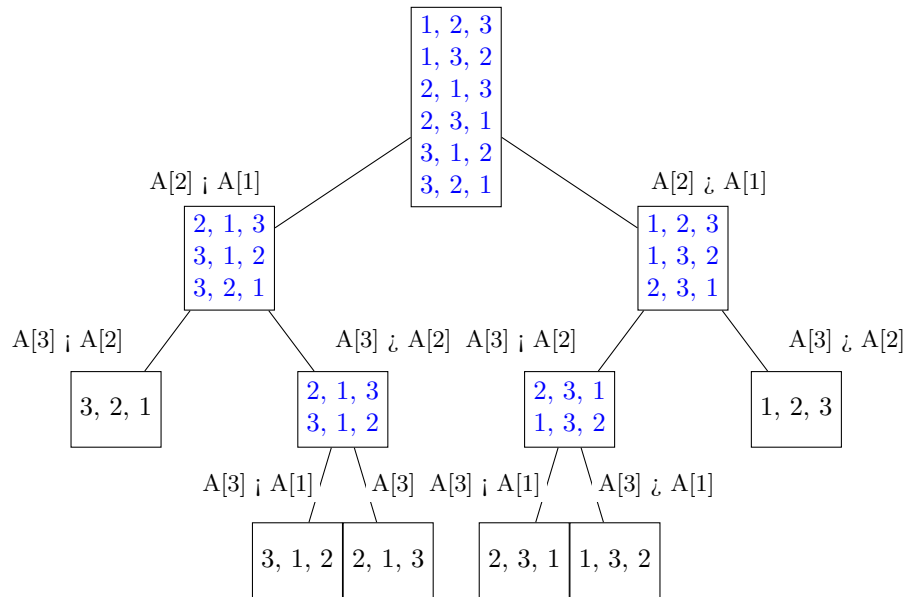
$\Rightarrow n \in \mathbb{N}$ mit $k := \lceil \log_2 n \rceil < 1 + \log_2 n$:

$$T(n) \leq T(2^k) \leq c(k+1)2^k < 2c(2 + \log_2 n)n = O(n \log n).$$

5.2 Satz zur Anzahl von Vergleichen

Satz 3. Jeder Sortieralgorithmus, der nur mit paarweisen Vergleichen von zwei Elementen arbeitet, benötigt zum Sortieren einer n -elementigen Menge mindestens $\log_2(n!)$ Vergleiche.

Der Beweis dieses Satzes lässt sich mit folgendem beispielhaften Vergleichsbaum für $n=3$ veranschaulichen:



Beweis: Gegeben: Eingabemenge $S = \{1, \dots, n\}$ mit einer totalen Ordnung
 Gesucht: einer der $n!$ Permutationen (Blätter im Vergleichsbaum). Jeder Vergleich halbiert die Anzahl an möglichen Permtationen (ein Vergleich ist ein Schritt im Baum). Ein Algorithmus braucht also k Vergleiche mit $2^k \geq n! \Rightarrow k \geq \log_2(n!)$.

Im Allgemeinen werden mehr Vegleiche gebraucht. Beispielweise werden für $n=5$ 7 Vergleiche gebraucht und nicht $\log_2(5!) = 6.9$ Vergleiche.

6 Quicksort

Im folgendem betrachten wir eine abgewandelte Form des mergesort.

- **Input** eine Menge $S = \{s_1; \dots; s_n\}$; eine durch Schlüssel induzierte partielle Ordnung $\preceq$ auf S .
- **Output** Berechne eine Bijektion $f: \{1, \dots, n\} \rightarrow S$ mit $f(j) \not\preceq f(i)$ für alle $1 \leq i < j \leq n$.

if $|S| > 1$
then

- wähle $x \in S$ beliebig
- $S_1 := \{s \in S \mid s \leq x\} \setminus \{x\}$
- $S_2 := \{s \in S \mid x \leq s\} \setminus \{x\}$
- $S_x := S \setminus (S_1 \cup S_2)$
- Sortiere S_1 und S_2 (soweit nicht leer) rekursiv mit Quicksort

– Füge die sortierten Mengen S_1 , S_x und S_2 aneinander

Ein Vorteil bei Quicksort ist, dass kein zusätzlicher temporärer Speicherplatz benötigt wird.

6.1 Laufzeit

Quicksort sortiert n Elemente in der Laufzeit $O(n^2)$.

Beweis: Es existiert ein $c \in \mathbb{N}$ mit $T(1) \leq c$. $T(n) \leq cn + \max_{0 \leq l \leq n-1} (T(l) + T(n-1-l))$ für alle $n \geq 2$.

Behauptung: $T(n) \leq cn^2$

Beweis per Induktion über n : Für $n = 1$ trivial.

$T(n) \leq \max_{0 \leq l \leq n-1} (cl^2 + c(n-1-l)^2) = cn + c(n-1)^2 < cn^2$

6.2 die Wahl des x

Beim Quicksort macht es einen großen Unterschied wie das x gewählt wird. Für kleine Listen oder schon relativ sortierte Listen macht es Sinn immer das erste Element der Liste als x zu wählen. Dann haben wir die oben bewiesene Laufzeit. Alternativ kann auch durch einen Zufallsalgorithmus ein zufälliges Element als x gewählt werden. Das scheint zwar erste einmal effektiver beläuft sich aber auf die selbe worst-case Laufzeit. Eine andere Möglichkeit wäre erst einmal den Median zu ermitteln und ihn als x zu wählen. Der Median kann in einer Zeit von $O(n)$ ermittelt werden und die Laufzeit würde sich dadurch auf $O(n \log n)$ verkürzen. Bei besonders großen Listen macht dies zwar manchmal Sinn, lohnt sich generell aber selten.

6.3 Erwartungswert

Der Erwartungswert der Laufzeit von Random-Quicksort für n Elemente ist $O(n \log(n))$.

Beweis: Es existiert ein $c \in \mathbb{N}$ mit $\bar{T}(1) \leq c$. $\bar{T}(n) \leq cn + \frac{1}{n} \sum_{i=1}^n (\bar{T}(i-1) + \bar{T}(n-i)) = cn + \frac{2}{n} \sum_{i=1}^{n-1} \bar{T}(i)$ für alle $n \geq 2$.

Behauptung: $\bar{T}(n) < 2cn(1 + \ln n)$.

Wir zeigen dies per Induktion über n . Für $n = 1$ ist die Aussage trivial. Für $n \geq 2$ erhalten wir mit der Induktionsvoraussetzung:

$$\begin{aligned} \bar{T}(n) &\leq cn + \frac{2}{n} \sum_{i=1}^{n-1} 2ci(1 + \ln i) \\ &\leq cn + \frac{2c}{n} \int_1^n 2x(1 + \ln x) dx \\ &= cn + \frac{2c}{n} \left[x^2(1 + \ln x) - \frac{x^2}{2} \right]_1^n \\ &= cn + 2cn(1 + \ln n) - cn - \frac{c}{n} < 2cn(1 + \ln n) \end{aligned}$$

Diese exzellente Laufzeit macht den Random-Quicksort relativ attraktiv.

7 Heapsort

Definiton 4. Ein **Heap** für eine endliche Menge S mit Schlüsseln $k : S \rightarrow K$ und einer totalen Ordnung $\leq$ von K ist eine Arboreszenz (ein gerichteter Graph) A mit einer Bijektion $f : V(A) \rightarrow S$, so dass die Heapordnung $k(f(v)) \leq k(f(w))$ für alle $(v, w) \in E(A)$ gilt. (Ein Baum, in dem jedes Elternteil einen kleineren Wert hat als seine Kinder)

Der Algorithmus, der so einen Heap erstellt arbeitet wie folgt: Beim Einfügen eines Elements in einen Heap mit bislang n Elementen wird dieses zunächst dem Knoten n zugeordnet. Anschließend wird das Element mit seinem Vorgänger verglichen und vertauscht solange bis der Vorgänger größer ist.

Beim Löschen eines Elements aus einem Heap mit n Elementen wird zunächst das n te Element an die Stelle des gelöschten Elements gesetzt und dann werden verschiedene Funktionen angewandt um die Heapordnung wiederherzustellen.

Beispiel für das Einfügen eines Elements:

